LoRA: Low-Rank Adaptation of Large Language Models

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen (ICLR 2022)

Abstract

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to specific tasks. As we pre-train larger models, full fine-tuning, which retrains all model parameters, becomes less feasible. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks.

1. Problem Statement

Full fine-tuning of GPT-3 175B requires storing 175 billion parameters and their optimizer states (e.g. Adam states requiring 3x parameters in FP32), incurring massive GPU memory overhead and deployment friction when serving multiple task-specific models.

2. Method: Low-Rank Decomposition
For a pre-trained weight matrix W_0 in R^{d x k}, we constrain its update Delta W by representing the latter with a low-rank decomposition:
*W = W_0 + Delta W = W_0 + (B * A) * (alpha / r)*
where B in R^{d x r}, A in R^{r x k}, and the rank r << min(d, k).

During training, W_0 is frozen and receives no gradient updates, while A and B contain trainable parameters. Note that both W_0 and Delta W = B * A are multiplied by the same input x:
*h = W_0 * x + Delta W * x = W_0 * x + (alpha / r) * B * A * x*

We use a Gaussian initialization for matrix A and zero initialization for matrix B, so Delta W = 0 at the beginning of training. We scale Delta W by alpha / r where alpha is a constant hyperparameter.

No Additional Inference Latency: When deployed in production, we can explicitly compute and store W = W_0 + B * A and serve requests normally. As a result, LoRA introduces zero additional inference latency compared to full fine-tuning.

3. Memory Efficiency and Experimental Results

Memory Footprint: When fine-tuning GPT-3 175B with Adam, LoRA reduces the VRAM consumption from 1.2 TB to 350 GB. With r = 4 and adapting query and value projection weights only, checkpoint size is reduced by roughly 10,000x (from 350GB to 35MB).

Performance: On GLUE and SuperGLUE benchmarks, LoRA matches or exceeds full fine-tuning performance on RoBERTa-large, DeBERTa-XXL, and GPT-3 175B while training as few as 0.01% of total model parameters.

Switching Tasks in Production: Serving multiple downstream tasks requires only swapping the small low-rank weight matrices A and B in GPU memory without reloading the massive 175B parameter base model.